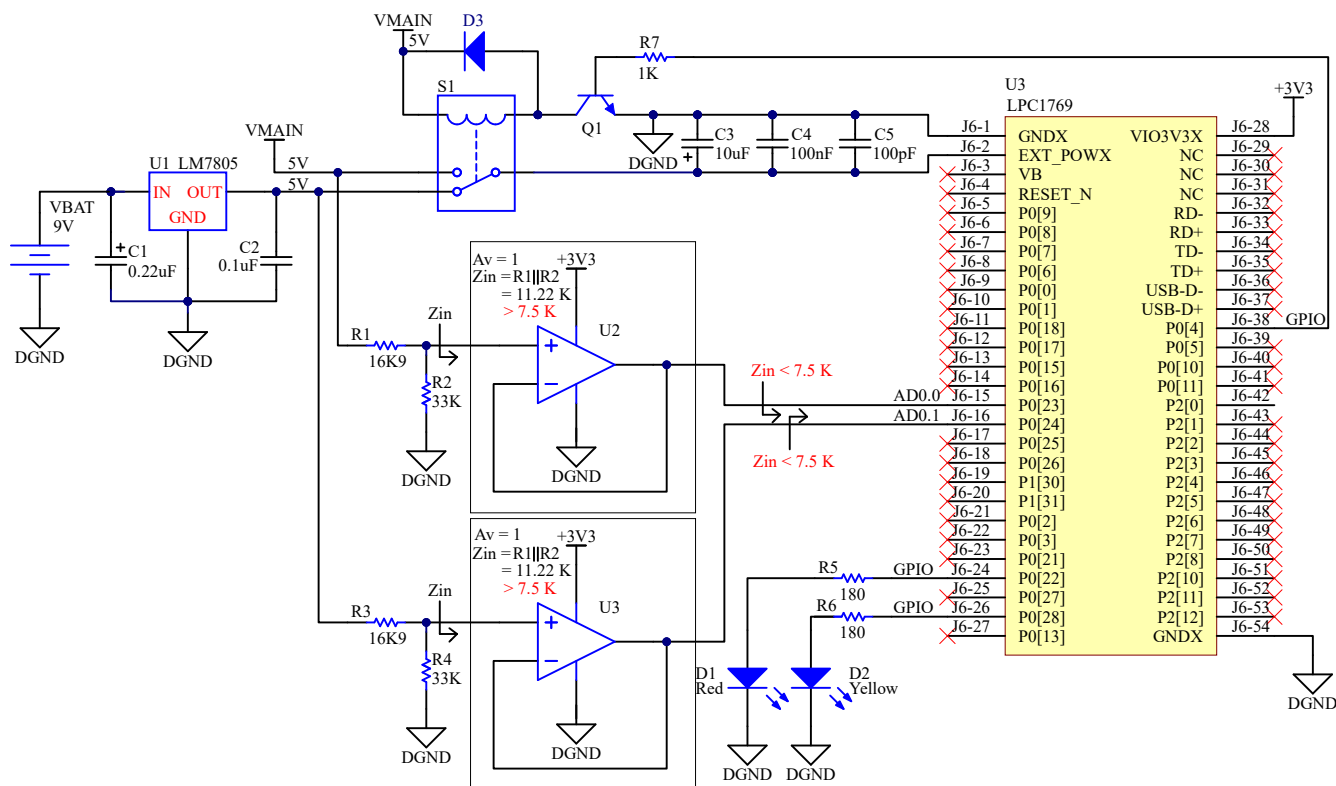


```

1  /*=====
2  * @file    Ejercicio_6_3_2.c
3  *
4  * @brief    Se requiere monitorizar la tensión de alimentación principal VMAIN
5  *           y la tensión de alimentación auxiliar VBAT mediante los canales
6  *           AD0.0 y AD0.1 del ADC, respectivamente.
7  *           Realizar la adquisición continua de ambas señales mediante el modo
8  *           Burst del ADC y gestionar la conmutación automática de alimentación.
9  *
10 * @details  - VMAIN se monitoriza mediante el canal AD0.0 del ADC.
11 *            - VBAT se monitoriza mediante el canal AD0.1 del ADC.
12 *            - Ambas tensiones se acondicionan mediante etapas de adaptación
13 *              analógica basadas en amplificadores operacionales.
14 *            - El acondicionamiento garantiza compatibilidad con el rango de
15 *              entrada del ADC del LPC1769 (0 V a 3.3 V) y mejora la inmunidad
16 *              al ruido durante la adquisición.
17 *            - Si VMAIN es menor que 5 V, se activa un rele de transferencia ubicado
18 *              en el pin P0.4 para conmutar la carga hacia VBAT y la alarma técnica
19 *              visual de prioridad media conformada por un LED amarillo en el
20 *              pin P0.22
21 *            - Si VBAT es menor que 3 V, se activa la alarma técnica visual de
22 *              prioridad alta conformada por un LED rojo en el pin P0.22, para
23 *              una condición de batería auxiliar baja.
24 *            - Implementación en alto nivel mediante drivers (NXP LPC17xx).
25 *
26 * @author   Ing. E. Migliore
27 *
28 * @version  1.0
29  *=====*/

```



$$k = \frac{V_{ADC}}{V_{IN}} = \frac{3V3}{5V} = 0.66$$

$$V_{ADC} = V_{IN} \left(\frac{R_2}{R_2 + R_1} \right) \Rightarrow R_1 = \frac{\left(1 - \frac{V_{ADC}}{V_{IN}} \right)}{\left(\frac{V_{ADC}}{V_{IN}} \right)} R_2 = \left(\frac{1 - k}{k} \right) R_2 = \left(\frac{1 - 0.66}{0.66} \right) * 33 \text{ k}\Omega \cong 16.9 \text{ k}\Omega$$

ADC electrical characteristics

$V_{DDA} = 2.5 \text{ V to } 3.6 \text{ V}$; $T_{amb} = -40 \text{ }^\circ\text{C to } +85 \text{ }^\circ\text{C}$ unless otherwise specified; ADC frequency 13 MHz; 12-bit resolution.[\[1\]](#)

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
V_{IA}	analog input voltage		0	-	V_{DDA}	V
C_{ia}	analog input capacitance		-	-	15	pF
R_{vsi}	voltage source interface resistance		[9] -	-	7.5	k Ω

```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5  #include "lpc17xx_gpio.h"
6  #include "lpc17xx_pinsel.h"
7  #include "lpc17xx_adc.h"
8  #include <stdbool.h>
9
10 /*=====
11 * DEFINICIÓN DE CONSTANTES
12 *=====*/
13 /* Parámetros del ADC */
14 #define ADC_VREF_MV          3300U
15 #define ADC_MAX_COUNTS      4095U
16 #define ADC_SAMPLE_RATE_HZ  200000U
17 /* Factor de reconstrucción del divisor de tensión */
18 /* k = 0.66 => 1/k = 50 / 33 */
19 #define VOLTAGE_DIVIDER_NUM  50U
20 #define VOLTAGE_DIVIDER_DEN  33U
21
22 /* Canales ADC */
23 #define VMAIN_ADC_CHANNEL    ADC_CHANNEL_0
24 #define VBAT_ADC_CHANNEL    ADC_CHANNEL_1
25
26 /* Pines de entrada analógica */
27 #define VMAIN_ADC_PORT      PINSEL_PORT_0
28 #define VMAIN_ADC_PIN      PINSEL_PIN_23
29 #define VBAT_ADC_PORT      PINSEL_PORT_0
30 #define VBAT_ADC_PIN      PINSEL_PIN_24
31
32 /* Pines de salida digital */
33 #define RELAY_PORT          PINSEL_PORT_0
34 #define RELAY_PIN           PINSEL_PIN_4
35 #define RELAY_PIN_MASK     (1U << 4)
36
37 #define ALARM_HIGH_PORT     PINSEL_PORT_0
38 #define ALARM_HIGH_PIN     PINSEL_PIN_22
39 #define ALARM_HIGH_PIN_MASK (1U << 22)
40
41 #define ALARM_MEDIUM_PORT   PINSEL_PORT_0
42 #define ALARM_MEDIUM_PIN   PINSEL_PIN_28
43 #define ALARM_MEDIUM_PIN_MASK (1U << 28)
44
45 /* Modalidades GPIO */
46 #define GPIO_INPUT          0U
47 #define GPIO_OUTPUT         1U
48
49 /* Umbrales de supervisión */
50 #define VMAIN_TRANSFER_TH_MV 5000U    // Umbral de transferencia VMAIN: 5 V
51 #define VBAT_LOW_TH_MV      3000U    // Umbral de batería auxiliar baja: 3 V
52
53 /*=====
54 * DEFINICION DE VARIABLES GLOBALES
55 *=====*/
56 volatile uint16_t vmain_raw    = 0;
57 volatile uint32_t vmain_adc_mv = 0;
58 volatile uint32_t vmain_mv     = 0;
59
60 volatile uint16_t vbat_raw     = 0;
61 volatile uint32_t vbat_adc_mv  = 0;
62 volatile uint32_t vbat_mv     = 0;
63
64 volatile bool vmain_data_ready = false;
65 volatile bool vbat_data_ready  = false;
66 volatile bool adc_flag_ready   = false;
67
68 /*=====
69 * PROTOTIPOS DE FUNCIONES

```

```

70  *=====*/
71  void cfgGPIO(void);
72  void cfgADC(uint32_t rate);
73
74  static uint32_t adcRawToMillivolts(uint16_t adc_raw);
75  static uint32_t adcMillivoltsToInputMillivolts(uint32_t adc_voltage_mv);
76  static void updatePowerSystem(void);
77
78  /*=====
79  * FUNCION PRINCIPAL
80  *=====*/
81  int main(void)
82  {
83      /*-----
84      * INICIALIZACION DEL MCU
85      *-----*/
86      cfgGPIO();
87      cfgADC(ADC_SAMPLE_RATE_HZ);
88
89      /*-----
90      * INICIO DEL PROGRAMA PRINCIPAL
91      *-----*/
92      while(1){
93          /*
94           * El procesamiento principal puede ejecutarse aquí.
95           * Las conversiones y la gestion de salidas se realizan por interrupción.
96           * La bandera indica que se proceso un nuevo par VMAIN/VBAT.
97           */
98          if (adc_flag_ready == true) {
99              adc_flag_ready = false;
100              // ...
101          }
102      }
103  }
104
105  /*=====
106  * DEFINICIONES DE FUNCIONES
107  *=====*/
108  /**
109   * @brief Configura las direcciones GPIO, modos de funcion y resistencias.
110   *
111   * @details - P0.4: Funcion GPIO como Salida Digital (Rele de Transferencia).
112   *           - P0.22: Funcion GPIO como Salida Digital (Alarma de prioridad alta)
113   *           - P0.28: Funcion GPIO como Salida Digital (Alarma de prioridad media)
114   *           - P0.23: Funcion AD0.0 como Entrada Analogica (VMAIN).
115   *           - P0.24: Funcion AD0.1 como Entrada Analogica (VBAT).
116   *
117   * @param None.
118   * @return None.
119   */
120  void cfgGPIO(void)
121  {
122      PINSEL_CFG_Type cfgPinRelay;
123      PINSEL_CFG_Type cfgPinAlarmHigh;
124      PINSEL_CFG_Type cfgPinAlarmMedium;
125      PINSEL_CFG_Type cfgPinVMAINvoltage;
126      PINSEL_CFG_Type cfgPinVBATvoltage;
127
128      // P0.4 como GPIO para el rele de transferencia
129      cfgPinRelay.Portnum = RELAY_PORT;
130      cfgPinRelay.Pinnum = RELAY_PIN;
131      cfgPinRelay.Funcnum = PINSEL_FUNC_0;
132      cfgPinRelay.Pinmode = PINSEL_PINMODE_PULLDOWN;
133      cfgPinRelay.OpenDrain = PINSEL_PINMODE_NORMAL;
134
135      // P0.22 como GPIO para alarma de prioridad alta
136      cfgPinAlarmHigh.Portnum = ALARM_HIGH_PORT;
137      cfgPinAlarmHigh.Pinnum = ALARM_HIGH_PIN;
138      cfgPinAlarmHigh.Funcnum = PINSEL_FUNC_0;

```

```

139     cfgPinAlarmHigh.Pinmode    = PINSEL_PINMODE_PULLDOWN;
140     cfgPinAlarmHigh.OpenDrain = PINSEL_PINMODE_NORMAL;
141
142     // P0.28 como GPIO para alarma de prioridad media
143     cfgPinAlarmMedium.Portnum  = ALARM_MEDIUM_PORT;
144     cfgPinAlarmMedium.Pinnum   = ALARM_MEDIUM_PIN;
145     cfgPinAlarmMedium.Funcnum  = PINSEL_FUNC_0;
146     cfgPinAlarmMedium.Pinmode  = PINSEL_PINMODE_PULLDOWN;
147     cfgPinAlarmMedium.OpenDrain = PINSEL_PINMODE_NORMAL;
148
149     // P0.23 como AD0.0 para monitorizar VMAIN
150     cfgPinVMAINvoltage.Portnum = VMAIN_ADC_PORT;
151     cfgPinVMAINvoltage.Pinnum  = VMAIN_ADC_PIN;
152     cfgPinVMAINvoltage.Funcnum = PINSEL_FUNC_1;
153     cfgPinVMAINvoltage.Pinmode = PINSEL_PINMODE_TRISTATE;
154     cfgPinVMAINvoltage.OpenDrain = PINSEL_PINMODE_NORMAL;
155
156     // P0.24 como AD0.1 para monitorizar VBAT
157     cfgPinVBATvoltage.Portnum  = VBAT_ADC_PORT;
158     cfgPinVBATvoltage.Pinnum   = VBAT_ADC_PIN;
159     cfgPinVBATvoltage.Funcnum  = PINSEL_FUNC_1;
160     cfgPinVBATvoltage.Pinmode  = PINSEL_PINMODE_TRISTATE;
161     cfgPinVBATvoltage.OpenDrain = PINSEL_PINMODE_NORMAL;
162
163     PINSEL_ConfigPin(&cfgPinRelay);
164     PINSEL_ConfigPin(&cfgPinAlarmHigh);
165     PINSEL_ConfigPin(&cfgPinAlarmMedium);
166     PINSEL_ConfigPin(&cfgPinVMAINvoltage);
167     PINSEL_ConfigPin(&cfgPinVBATvoltage);
168
169     GPIO_SetDir(RELAY_PORT, RELAY_PIN_MASK, GPIO_OUTPUT);
170     GPIO_SetDir(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK, GPIO_OUTPUT);
171     GPIO_SetDir(ALARM_MEDIUM_PORT, ALARM_MEDIUM_PIN_MASK, GPIO_OUTPUT);
172
173     // Estado inicial seguro: relé activado (VMAIN) y LEDs desactivados
174     GPIO_SetValue(RELAY_PORT, RELAY_PIN_MASK);
175     GPIO_ClearValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
176     GPIO_ClearValue(ALARM_MEDIUM_PORT, ALARM_MEDIUM_PIN_MASK);
177
178     return;
179 }
180
181 /*****
182  * @brief   Configura e inicializa el periférico ADC en modo Burst.
183  *
184  * @details Habilita los canales AD0.0 y AD0.1 para su conversion continua.
185  *           En modo Burst, el ADC convierte secuencialmente los canales
186  *           seleccionados sin requerir disparos por software ni por Timer.
187  *
188  * @param   <rate> Tasa de conversion del ADC [S/s].
189  *
190  * @return  None.
191  *****/
192 void cfgADC(uint32_t rate)
193 {
194     ADC_Init(LPC_ADC, rate);
195
196     ADC_ChannelCmd(LPC_ADC, VMAIN_ADC_CHANNEL, ENABLE);
197     ADC_ChannelCmd(LPC_ADC, VBAT_ADC_CHANNEL, ENABLE);
198
199     ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);
200     ADC_IntConfig(LPC_ADC, ADC_ADINTEN1, ENABLE);
201
202     ADC_BurstCmd(LPC_ADC, ENABLE);
203
204     NVIC_EnableIRQ(ADC_IRQn);
205
206     return;
207 }

```

```

208
209 /*****
210  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector ADC.
211  *
212  * @details Implementa una secuencia continua de adquisicion en modo Burst:
213  *           1. Verifica y lee el resultado de AD0.0 correspondiente a VMAIN.
214  *           2. Verifica y lee el resultado de AD0.1 correspondiente a VBAT.
215  *           3. Convierte ambos resultados a sus tensiones de entrada [mV].
216  *           4. Procesa la transferencia y las alarmas cuando se dispone de un
217  *              nuevo resultado de ambos canales.
218  *
219  * @param   None.
220  * @return  None.
221  *****/
222 void ADC_IRQHandler(void)
223 {
224     /* Lectura y acondicionamiento de VMAIN */
225     if (ADC_ChannelGetStatus(LPC_ADC, VMAIN_ADC_CHANNEL, ADC_DATA_DONE) == SET){
226         vmain_raw = ADC_ChannelGetData(LPC_ADC, VMAIN_ADC_CHANNEL);
227         vmain_adc_mv = adcRawToMillivolts(vmain_raw);
228         vmain_mv = adcMillivoltsToInputMillivolts(vmain_adc_mv);
229         vmain_data_ready = true;
230     }
231
232     /* Lectura y acondicionamiento de VBAT */
233     if (ADC_ChannelGetStatus(LPC_ADC, VBAT_ADC_CHANNEL, ADC_DATA_DONE) == SET){
234         vbat_raw = ADC_ChannelGetData(LPC_ADC, VBAT_ADC_CHANNEL);
235         vbat_adc_mv = adcRawToMillivolts(vbat_raw);
236         vbat_mv = adcMillivoltsToInputMillivolts(vbat_adc_mv);
237         vbat_data_ready = true;
238     }
239
240     /* Procesamiento de un par nuevo de mediciones */
241     if ((vmain_data_ready == true) && (vbat_data_ready == true)) {
242         updatePowerSystem();
243
244         vmain_data_ready = false;
245         vbat_data_ready = false;
246         adc_flag_ready = true;
247     }
248 }
249
250 /*****
251  * @brief   Convierte el resultado digital del ADC a milivoltios.
252  *
253  * @param   <adc_raw> Resultado digital del ADC de 12 bits.
254  *
255  * @return  Tension presente en la entrada del ADC, expresada en milivoltios.
256  *****/
257 static uint32_t adcRawToMillivolts(uint16_t adc_raw)
258 {
259     return (((uint32_t)adc_raw * ADC_VREF_MV) / ADC_MAX_COUNTS);
260 }
261
262 /*****
263  * @brief   Compensa la atenuación del divisor resistivo de entrada.
264  *
265  * @details Calcula la tension monitorizada antes del divisor resistivo:
266  *
267  *           VIN = VADC * (VOLTAGE_DIVIDER_NUM / VOLTAGE_DIVIDER_DEN)
268  *
269  * @param   <adc_voltage_mv> Tensión medida en la entrada del ADC [mV].
270  *
271  * @return  Tension de entrada antes del divisor, expresada en milivoltios.
272  *****/
273 static uint32_t adcMillivoltsToInputMillivolts(uint32_t adc_voltage_mv)
274 {
275     return ((adc_voltage_mv * VOLTAGE_DIVIDER_NUM) / VOLTAGE_DIVIDER_DEN);
276 }

```